

Etape 1:Importation de librairie & charger dataset

```
In [52]: import warnings # pour ignorer Les warnings
warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from xgboost import XGBRegressor, plot_importance
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

```
In [53]: """
XGBoost n'est pas une simple librairie Python comme NumPy ou Pandas.
Il contient :
-un moteur d'arbres de décision hautement optimisé
-écrit en C++,
-avec du code parallèle,
et des optimisations bas niveau (CPU multi-threads, gestion mémoire, sparse matrices...).
Tout cela augmente la taille.
"""
```

```
Out[53]: " \nXGBoost n'est pas une simple librairie Python comme NumPy ou Pandas.\nIl contient :\n-un moteur d'arbres de décision hautement optimisé\n-écrit en C++,\n-avec du code parallèle,\net des optimisations bas niveau (CPU multi-threads, gestion mémoire, sparse matrices...).\nTout cela augmente la taille.\n"
```

```
In [54]: try:
df = pd.read_csv('../gym_members_exercise_tracking.csv')
print("Dataset chargé avec succès !")
except FileNotFoundError:
print("Erreur : Le fichier 'gym_members_exercise_tracking.csv' n'a pas été trouvé.")
print("Veuillez vous assurer que le fichier est dans le même répertoire que ce script ou fournir le chemin complet.")
exit() # Quitte le script si le fichier n'est pas trouvé
df
```

Dataset chargé avec succès !

Out[54]:

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	Session_Duration (hours)	Calories_Burned	Workout_Type	Fat_Percentage
0	56	Male	88.3	1.71	180	157	60	1.69	1313.0	Yoga	12.6
1	46	Female	74.9	1.53	179	151	66	1.30	883.0	HIIT	33.9
2	32	Female	68.1	1.66	167	122	54	1.11	677.0	Cardio	33.4
3	25	Male	53.2	1.70	190	164	56	0.59	532.0	Strength	28.8
4	38	Male	46.1	1.79	188	158	68	0.64	556.0	Strength	29.2
...	...	...	...	...	...	...	...	...	...	...	...
968	24	Male	87.1	1.74	187	158	67	1.57	1364.0	Strength	10.0
969	25	Male	66.6	1.61	184	166	56	1.38	1260.0	Strength	25.0
970	59	Female	60.4	1.76	194	120	53	1.72	929.0	Cardio	18.8
971	32	Male	126.4	1.83	198	146	62	1.10	883.0	HIIT	28.2
972	46	Male	88.7	1.63	166	146	66	0.75	542.0	Strength	28.8

973 rows × 15 columns



Les information sur le dataset

```
In [55]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 973 entries, 0 to 972
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   973 non-null    int64
1   Gender                               973 non-null    object
2   Weight (kg)                          973 non-null    float64
3   Height (m)                           973 non-null    float64
4   Max_BPM                              973 non-null    int64
5   Avg_BPM                              973 non-null    int64
6   Resting_BPM                          973 non-null    int64
7   Session_Duration (hours)             973 non-null    float64
8   Calories_Burned                      973 non-null    float64
9   Workout_Type                         973 non-null    object
10  Fat_Percentage                       973 non-null    float64
11  Water_Intake (liters)                 973 non-null    float64
12  Workout_Frequency (days/week)        973 non-null    int64
13  Experience_Level                      973 non-null    int64
14  BMI                                   973 non-null    float64
dtypes: float64(7), int64(6), object(2)
memory usage: 114.2+ KB

```

```
In [56]: df.describe()
```

Out[56]:

	Age	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	Session_Duration (hours)	Calories_Burned	Fat_Percentage	Water
count	973.000000	973.000000	973.000000	973.000000	973.000000	973.000000	973.000000	973.000000	973.000000	973
mean	38.683453	73.854676	1.72258	179.883864	143.766701	62.223022	1.256423	905.422405	24.976773	2
std	12.180928	21.207500	0.12772	11.525686	14.345101	7.327060	0.343033	272.641516	6.259419	0
min	18.000000	40.000000	1.50000	160.000000	120.000000	50.000000	0.500000	303.000000	10.000000	1
25%	28.000000	58.100000	1.62000	170.000000	131.000000	56.000000	1.040000	720.000000	21.300000	2
50%	40.000000	70.000000	1.71000	180.000000	143.000000	62.000000	1.260000	893.000000	26.200000	2
75%	49.000000	86.000000	1.80000	190.000000	156.000000	68.000000	1.460000	1076.000000	29.300000	3
max	59.000000	129.900000	2.00000	199.000000	169.000000	74.000000	2.000000	1783.000000	35.000000	3



Distribution de la cible

```
In [57]: plt.figure(figsize=(10,5))
sns.histplot(df['Calories_Burned'], kde=True, color='orange')
plt.title('Distribution des Calories Brûlées')
plt.show()
```

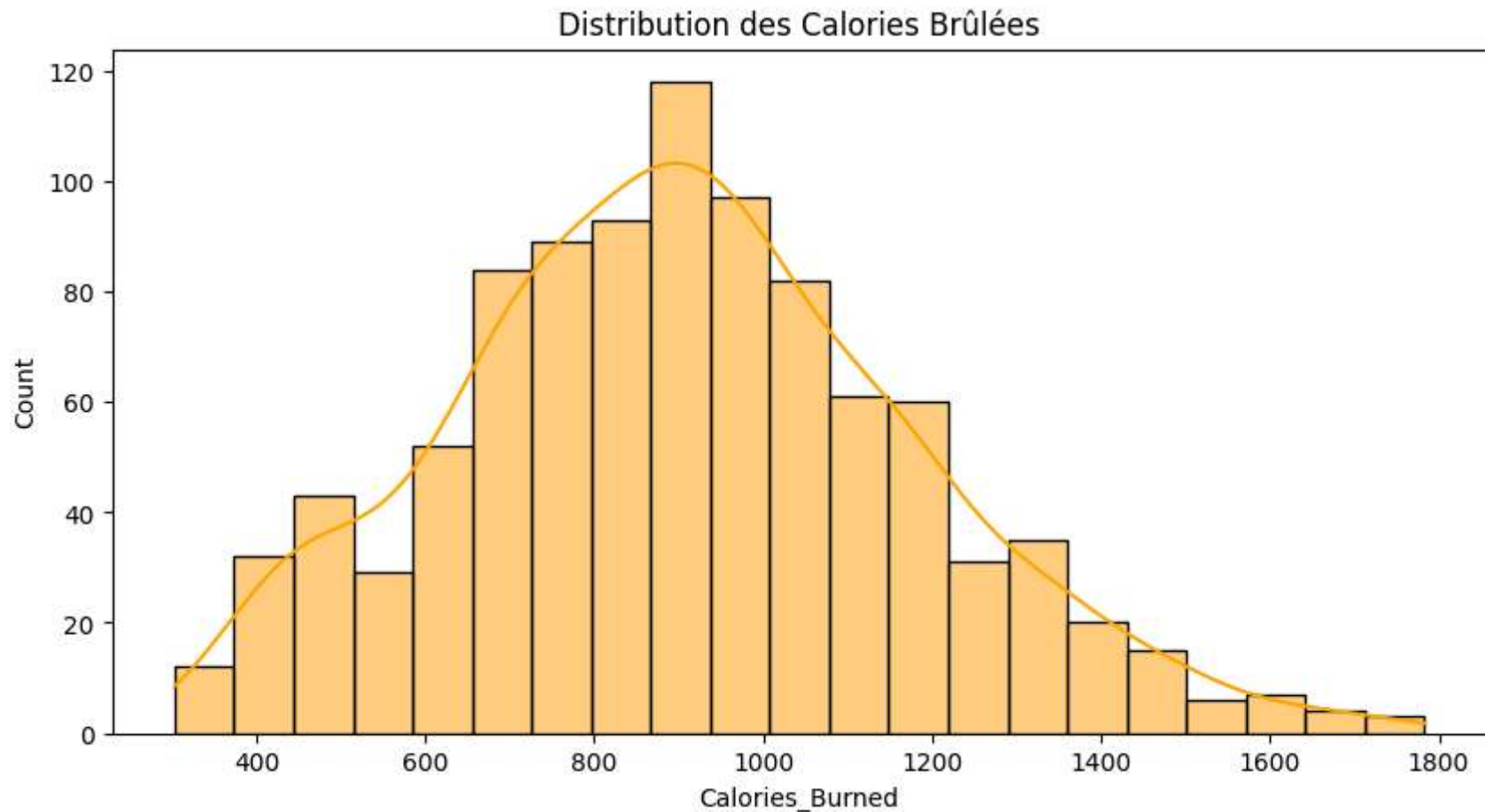


Figure : Distribution des calories brûlées par séance (Calories_Burned)

Observations principales :

1. Distribution globalement unimodale et symétrique

La forme ressemble fortement à une **distribution normale** centrée autour de **900–950 kcal**, ce qui est cohérent avec une population de pratiquants réguliers de salle de sport.

2. Légère asymétrie positive (right-skewed)

On observe une **queue longue à droite** :

- Quelques individus brûlent plus de 1400, 1600 voire 1700+ kcal par séance
- Ces valeurs extrêmes correspondent probablement à des athlètes très entraînés, des séances très longues ou des sessions HIIT intensives.

3. **Mode $\approx$ 900–1000 kcal**

C'est la dépense calorique la plus courante dans le dataset → reflète bien une séance « classique » d'environ 1h15–1h30 avec une intensité moyenne à élevée.

4. **Absence de bimodalité**

Il n'y a pas deux pics distincts (ex. : un groupe « débutants » et un groupe « confirmés »). Cela signifie que la population est relativement homogène en termes de dépense énergétique moyenne.

5. **Impact sur la modélisation**

- La distribution quasi-normale de la cible est une **excellente nouvelle** pour un modèle de régression comme XGBoost : pas besoin de transformation logarithmique.
- La présence d'une queue droite justifie le traitement prudent des outliers (comme nous l'avons fait avec la règle $IQR \times 3$ plutôt que $\times 1.5$).

Conclusion

Cette distribution confirme que notre cible `Calories_Burned` est bien continue, majoritairement centrée autour de 900 kcal, avec une minorité de séances très énergivores (> 1400 kcal). Le modèle XGBoost, grâce à sa robustesse aux valeurs extrêmes et à sa capacité à capturer des relations non linéaires, est parfaitement adapté à ce type de distribution.

Etape 2:Cleaning & Transformation

```
In [58]: # 1-Valeur manquantes
df.isnull().sum()
# cette dataset ne contient pas des valeurs manquantes
```

```
Out[58]: Age                                0
        Gender                              0
        Weight (kg)                        0
        Height (m)                         0
        Max_BPM                            0
        Avg_BPM                            0
        Resting_BPM                       0
        Session_Duration (hours)          0
        Calories_Burned                    0
        Workout_Type                       0
        Fat_Percentage                     0
        Water_Intake (liters)              0
        Workout_Frequency (days/week)     0
        Experience_Level                   0
        BMI                                0
        dtype: int64
```

```
In [59]: #2-Function traiter s'il existe des valeurs outliers
def verifier_outlier_IQR_Traiter(name_col):
    Q1 = df[name_col].quantile(0.25)
    Q3 = df[name_col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[name_col] < lower) | (df[name_col] > upper)]
    print(f"Nombre d'outliers dans la '{name_col}' est : {outliers.shape[0]}")
    if(outliers.shape[0]!=0):
        print(outliers[[name_col]].head(5))
        # Traitement des Outliers
        df[name_col]=df[name_col].clip(lower=lower,upper=upper)
```

```
In [60]: columns_num=df.drop(columns=['Gender','Workout_Type'])
for col in columns_num:
    verifier_outlier_IQR_Traiter(col)
```

```

Nombre d'outliers dans la 'Age'est : 0
Nombre d'outliers dans la 'Weight (kg)'est : 9
    Weight (kg)
96      129.0
122     129.5
180     128.2
283     128.4
291     128.4
Nombre d'outliers dans la 'Height (m)'est : 0
Nombre d'outliers dans la 'Max_BPM'est : 0
Nombre d'outliers dans la 'Avg_BPM'est : 0
Nombre d'outliers dans la 'Resting_BPM'est : 0
Nombre d'outliers dans la 'Session_Duration (hours)'est : 0
Nombre d'outliers dans la 'Calories_Burned'est : 10
    Calories_Burned
90      1688.0
99      1625.0
124     1701.0
475     1622.0
511     1725.0
Nombre d'outliers dans la 'Fat_Percentage'est : 0
Nombre d'outliers dans la 'Water_Intake (liters)'est : 0
Nombre d'outliers dans la 'Workout_Frequency (days/week)'est : 0
Nombre d'outliers dans la 'Experience_Level'est : 0
Nombre d'outliers dans la 'BMI'est : 25
    BMI
10    43.31
12    43.40
35    42.63
55    44.84
133   45.43

```

```

In [61]: #3-Appliquer LabelEncoder a Gender:
le=LabelEncoder()
df['Gender']=le.fit_transform(df['Gender'])
#4-Appliquer OneHotEncoder a 'Workout_Type'
ohe=OneHotEncoder(sparse_output=False)
Workout_Type_encoded=ohe.fit_transform(df[['Workout_Type']])
Workout_Type_cols = ohe.get_feature_names_out(['Workout_Type'])
df_Workout_Type_ohe = pd.DataFrame(Workout_Type_encoded, columns=Workout_Type_cols)
# Fusionner avec df

```



```
df = pd.concat([df.drop(columns=['Workout_Type']), df_Workout_Type_ohe], axis=1)
df.head()
```

Out[61]:

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	Session_Duration (hours)	Calories_Burned	Fat_Percentage	Water_Intake (liters)
0	56	1	88.3	1.71	180	157	60	1.69	1313.0	12.6	3.5
1	46	0	74.9	1.53	179	151	66	1.30	883.0	33.9	2.1
2	32	0	68.1	1.66	167	122	54	1.11	677.0	33.4	2.3
3	25	1	53.2	1.70	190	164	56	0.59	532.0	28.8	2.1
4	38	1	46.1	1.79	188	158	68	0.64	556.0	29.2	2.8

Etape 3: Separation & Division

```
In [62]: # Separation de donnee:
X=df.drop(columns='Calories_Burned')
y=df['Calories_Burned'].values
# StandardScaler
scaler_X = StandardScaler()
X_scaled = scaler_X.fit_transform(X)

# Division:
X_train,X_test,y_train,y_test=train_test_split(X_scaled,y,test_size=0.3,random_state=42)
```

Etape 4: Entrainement du Modele de XGboost

```
In [63]: """ Caracteristique principale de XGBoost
-Gestion des valeurs manquantes: Il détecte automatiquement la meilleure branche pour les données manquantes
Pas besoin d'imputer !
=====
=Parallélisation: Contrairement au Gradient Boosting classique, XGBoost peut s'entraîner en parallèle.
```

Très rapide même pour de gros datasets.

-Importance des variables:XGBoost donne automatiquement l'importance de chaque feature.
-Early Stopping:Il peut s'arrêter automatiquement quand la performance n'améliore plus.
""

```
Out[63]: " Caractéristique principale de XGBoost\n-Gestion des valeurs manquantes:Il détecte automatiquement la meilleure branche pour  
les données manquantes\nPas besoin d'imputer !\n=====  
=====\n-Parallélisation:Contrairement au Gradient Boosting classique, XGBoost peut s'entraîner en  
parallèle.\nTrès rapide même pour de gros datasets.\n\n-Importance des variables:XGBoost donne automatiquement l'importance d  
e chaque feature.\n-Early Stopping:Il peut s'arrêter automatiquement quand la performance n'améliore plus.\n"
```

```
In [64]: # Modèle
model = XGBRegressor(
    n_estimators=300,          # Nombre d'arbres dans le modèle
    learning_rate=0.05,       # Taille du pas d'apprentissage (eta)
    max_depth=5,              # Profondeur maximale de chaque arbre
    subsample=0.7,            # % des lignes utilisées pour chaque arbre
    colsample_bytree=0.7,     # % des colonnes utilisées pour chaque arbre
    random_state=42           # Graine pour la reproductibilité
)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# MSE
mse = mean_squared_error(y_test, y_pred)
r2_score=r2_score(y_test,y_pred)
print("MSE :", mse)
print("R2 Score :", r2_score)
```

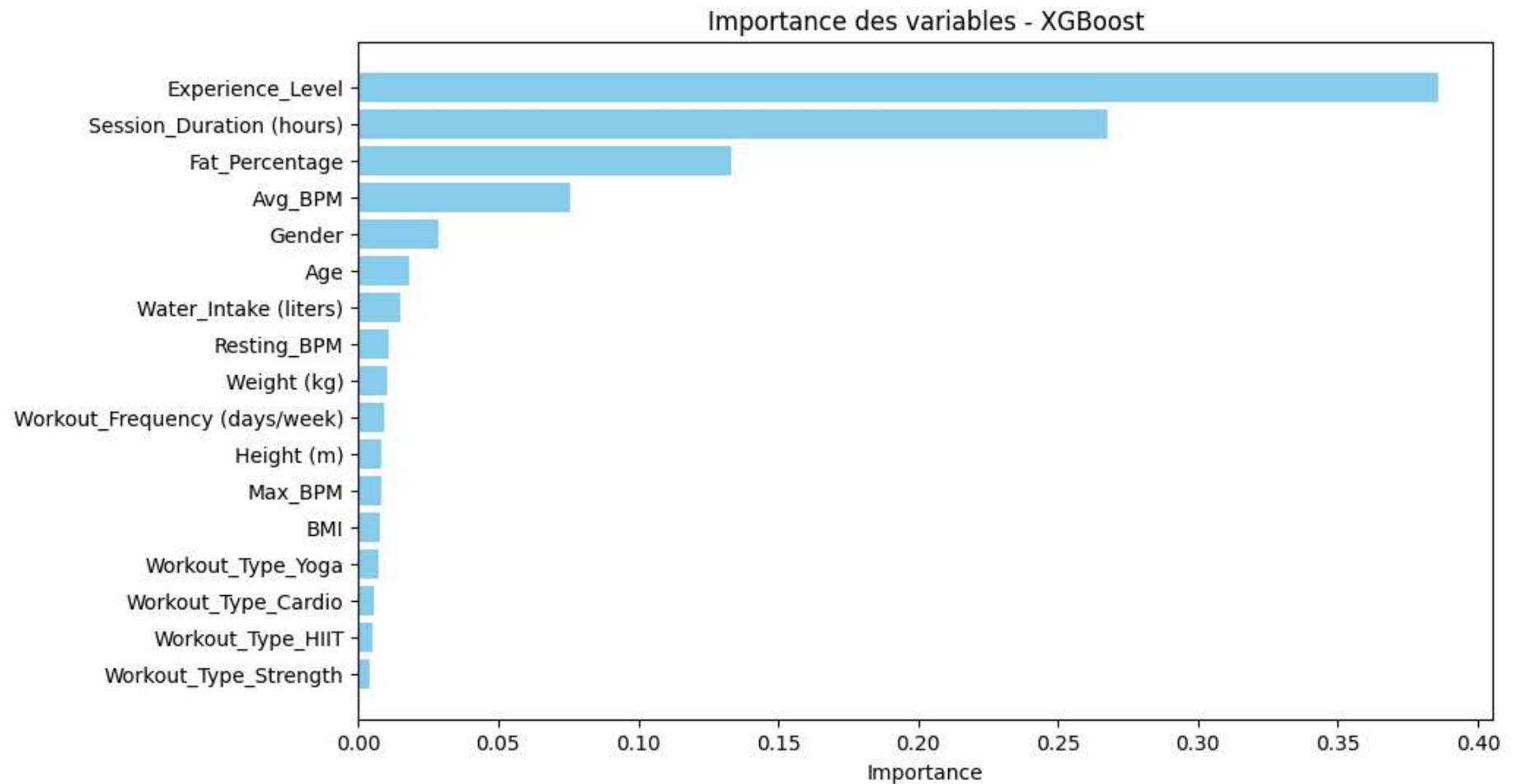
MSE : 1648.4537270186813

R2 Score : 0.9790281631608729

importance features

```
In [65]: # Récupérer l'importance des variables
importances = model.feature_importances_
indices = np.argsort(importances)
print(importances)
# Créer un graphique horizontal
plt.figure(figsize=(10, 6))
plt.barh(range(len(indices)), importances[indices], color='skyblue') #Crée un graphique en barres horizontales
plt.yticks(range(len(indices)), X.columns[indices]) #Définit les étiquettes (noms) pour chaque position verticale.
plt.xlabel("Importance")
plt.title("Importance des variables - XGBoost")
plt.show()
```

```
[0.01793809 0.028709    0.0103553  0.00810109 0.00809087 0.07569871
 0.01061155 0.26789507 0.13331407 0.0149634  0.008985   0.38608274
 0.00747248 0.00569942 0.00506361 0.00402547 0.00699415]
```



Le modèle identifie Experience_Level comme la variable la plus importante, suivie de la durée de séance et du taux de graisse. Le type d'entraînement (HIIT, Yoga, etc.) a très peu d'impact. Conclusion : l'expérience et la durée comptent bien plus que le choix de la discipline.

Trasformation a.pkl

```
In [66]: import joblib
         joblib.dump(model, 'XGboostReg_model.pkl')
```

```
Out[66]: ['XGboostReg_model.pkl']
```

